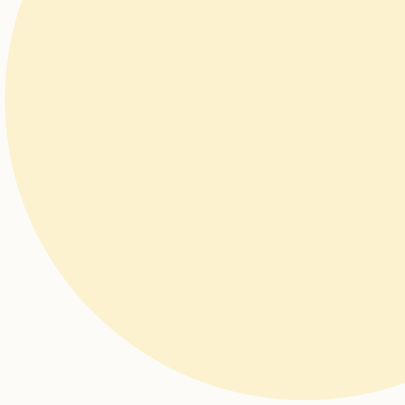




TypeScript

Interfaces & Type Aliases

The Rulebooks of Code [cite: 6]

Planrs Academy

Mastering Structured Data for Grade 6

The Messy Digital Library Challenge [cite: 13]

Imagine you have a huge digital library, but all the books are mixed up and messy! [cite: 14]

How can we organize our digital world like a clean, neat cupboard? [cite: 15]

Without rules, our code becomes like a messy shelf where books are upside down and out of place. This makes it very hard for the computer (and us!) to find what we need.

Today we will learn the tools that help us keep our code organized and tidy!
[cite: 16]

Building Blocks [cite: 19]

We already learned about strings, numbers, and booleans—the simple building blocks of code. [cite: 20]

These basic types help TypeScript understand what kind of data we are working with! [cite: 21]

- **Name:** `string` stores text like "Priya" or "Arjun" [cite: 22]
- **Age:** `number` stores numbers like 11 or 12 [cite: 23]

But what happens when we want to store a whole profile? Just one building block isn't enough!

WHAT IS AN INTERFACE?

The Blueprint [cite: 27]

An Interface is like a blueprint or a rulebook for an object. [cite: 28]

Think of it like an Aadhaar card: Every Aadhaar card must have a Name, Address, and a 12-digit Number. [cite: 29]

If any of these are missing, the card is incomplete! [cite: 30]

Interfaces define the **Rules** for what information must be there. [cite: 31] If a property is missing, TypeScript will tell you! [cite: 32]

Student Profile Rulebook [cite: 45]

This is the "rulebook" for a Student—it tells TypeScript exactly what properties every student object must have! [cite: 49]

```
interface Student {  
  name: string; [cite: 46]  
  age: number; [cite: 47]  
  grade: number; [cite: 48]  
}
```

By creating this interface, we ensure that every student we create follows the same structure.

Organized Code keeps it correct! [cite: 57]

Interfaces help keep our code neat—like organizing your school cupboard!
[cite: 59]

Messy Code (X) [cite: 53]

```
let student1 = {  
  nam: "Priya",  
  age: "twelve"  
}  
// Missing grade, wrong types! [cite: 54]
```

Organized Code [cite: 55]

```
let student1: Student = {  
  name: "Priya",  
  age: 12,  
  grade: 6  
} [cite: 56]
```

Rulebook Flexibility [cite: 60]

Some information is mandatory, like Student Name and Parent Phone Number. These fields **MUST** be filled in! [cite: 61, 63, 64]

Other information can be optional, like an Email ID. You can skip it if you don't have one. [cite: 65, 67, 68]

The Question Mark (?):

studentName: string - Required field [cite: 69]

emailID?: string - Optional field [cite: 70]

The Nickname [cite: 73]

A Type Alias is like a pet name or nickname for a type! [cite: 74]

Instead of writing "a sequence of numbers representing a pincode," we simply call it **Pincode**. [cite: 75]

```
type Pincode = string; [cite: 76]
```

Now whenever we need a pincode, we just use our nickname "Pincode" instead of explaining it every time! [cite: 77]

Union Types [cite: 83]

A Type Alias with Union Types lets one nickname represent multiple possible types. [cite: 83]

This makes our code flexible and easy to read! [cite: 84]

```
type ID = number | string; [cite: 86]
```

This means an ID can be either a **number** (like 12345) or a **string** (like "STU-001"). [cite: 87]

Union types use the `|` symbol to combine options. [cite: 89]

Cricket Player Stats Interface [cite: 96]

We can organize cricket player data using interfaces and type alias nicknames together! [cite: 104]

```
interface CricketPlayer { [cite: 98]
  name: string; [cite: 99]
  runs: number; [cite: 100]
  wickets?: number; [cite: 101]
  team: string; [cite: 102]
  playerId: ID; [cite: 103]
}
```

The playerId uses our **ID** alias from the previous page. They work as a team! [cite: 104, 116]

Activity: Build Your Own Interface! [cite: 110]

Design an interface for a **Cricket Player Card** or a **Zomato Order Summary**. [cite: 110]

Think about what information each one needs! [cite: 111]

Cricket Player Card: Name (string), Runs (number), Wickets (optional number), Team (string). [cite: 111]

Zomato Order: Dish Name (string), Price (number), IsVeg (boolean). [cite: 112]

When to Use Each One? [cite: 113]

- **Interfaces:** Use to create the blueprint or rulebook for objects. They define the **structure**! [cite: 114, 117]
- **Type Aliases:** Use as **nicknames** for simple or complex types to simplify names. [cite: 115, 118]

Remember: Organizing code keeps it neat, just like organizing your school cupboard! [cite: 120]

Thank you for learning with us at Planrs Academy! [cite: 121, 122]